GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 6 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Intervals and scheduling

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Intervals and scheduling | 5 questions | tap a title to jump to it

M	1. Meeting Rooms / Interval Scheduling Variants	1Point3Acres	2026-09-29
M	2. Compute minimum number of rooms needed	PracHub	2026-02-12
M	3. Union of Sorted Interval Lists	1Point3Acres	2026-08-16
M	4. Solve Rooms and Top-K Streams	PracHub	2026-06-05
M	5. Compute servers needed for daily recurring jobs	PracHub	2025-11-05

STUDY PATH: question 39 of 167 (Core 90)

Priority score 11.32 | Intervals and scheduling | asked 21 days ago | frequency Very high

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 1 of 5

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 44/228**

ABOUT THIS QUESTION

Topic: Core 90 - Intervals and scheduling

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

QUESTION

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

QUESTION

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

APPROACH

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

HOW TO THINK ABOUT IT

The number of rooms needed is the largest number of intervals overlapping at any moment. Turn each interval into +1 at its start and -1 at its end, sort the events (ends before starts at the same time) and keep a running count. To say which room each meeting gets, keep a min-heap of (end time, room) and reuse the room that frees up first.

Basis: LeetCode 253, Meeting Rooms II, with the common variants: room assignment and cyclic (24-hour) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
```

```python
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.
#
# BRUTE FORCE (say it first): Count the overlap at every
# start time
# For every interval count how many intervals contain its
# start point and take the maximum. Correct and simple;
# quadratic. Time O(n^2). Space O(1).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count the overlap at every
```

```python
# start time | time O(n^2)
def min_rooms_brute(intervals):
    return max(
        (
            sum(
                1
                for s2, e2 in intervals
                if s2 <= s < e2
            )
            for s, e in intervals
        ),
        default=0,
    )


# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
```

```python
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms
```

```python
# VARIANT: on-call consolidation
def oncall_segments(rotations):
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out


# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# time).
# Answer: Split every wrapped meeting [s, e) with s >= e
# into [s, 24h) and [0, e) and run the same sweep: the
# overlap at any clock time is unchanged. A meeting that
# is exactly 24 hours long covers the whole day.
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
            parts.append((s, e))
        else:
            # wraps past midnight: split at the boundary
```

```python
            parts += [(s, day), (0, e)]
    return min_rooms(parts)


if __name__ == "__main__":
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
    gap = [("A", 1, 2), ("B", 4, 5)]
    assert len(oncall_segments(gap)) == 2
```

NOTES FROM CANDIDATES

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.

The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 40 of 167 (Core 90)

Priority score 5.32 | Intervals and scheduling | asked 250 days ago

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 2 of 5

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 42/228

ABOUT THIS QUESTION

Topic: Core 90 - Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

- Input: `[[0,30],[5,10],[15,20]]`
- Output: `2` (One room can host `[0,30]`, the other can host `[5,10]` then `[15,20]`.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

QUESTION

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and
    [15,20], so at most 2 rooms are needed at the
    same time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

APPROACH

Sort by start, min-heap of end times (reuse a room when earliest end <= start), or two-pointer sweep on sorted starts/ends. O(n log n).

HOW TO THINK ABOUT IT

The rooms needed equal the largest number of meetings that overlap at one moment. Two ways to find that without checking every moment: sort by start and keep a min-heap of the end times of rooms in use (reuse the room that frees up first), or sort starts and ends separately and sweep with two pointers.

Basis: LeetCode 253, Meeting Rooms II (half-open intervals [start, end): a meeting ending at t does not conflict with one starting at t).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION · Sort by start with a min-heap of end times

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

FOLLOW-UP 2 Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

FOLLOW-UP 3 Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimum number of rooms needed
#
# ## Problem You are given a list of meetings, each with a
# start time and end time. A single room can host only one
# meeting at a time. Two meetings **overlap** if one
# starts before the other ends (treat meetings as half-
# open intervals **[start, end)** so a meeting ending at
# time t does not conflict with another starting at time
# t). ### Task Return the **minimum number of rooms**
# required to host all meetings. ### Input - intervals: an
# array of n pairs [start, end] where 0 <= start <= end.
# ### Output - An integer: the minimum number of rooms
# needed. ### Constraints (typical interview bounds) - 1
# <= n <= 200000 - Times are integers in a reasonable
# range (e.g., 0 . 10^9).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can one person attend all meetings (LeetCode 252)?
# 2. Return which room each meeting uses.
# 3. Meetings arrive online.
#
# INTERVIEW NOTES
# The rooms needed equal the largest number of meetings
# that overlap at one moment. Two ways to find that
# without checking every moment: sort by start and keep a
# min-heap of the end times of rooms in use (reuse the
# room that frees up first), or sort starts and ends
# separately and sweep with two pointers.
#
# BRUTE FORCE (say it first): Count the overlap at every
# time unit
# For each integer time count the meetings with start <= t
# < end and take the maximum. Correct and the oracle
# (integer times); it depends on the length of the time
# range, not only on n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Count the overlap at every
# time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals) + 1,
        )
    )


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)


# FOLLOW-UP 1: Can one person attend all meetings
# (LeetCode 252)?
# Answer: Sort by start and check that each meeting ends
# no later than the next one starts. O(n log n), O(1)
# extra.
def can_attend_all(intervals):
    # LeetCode 252: one person, no overlap at all
    ordered = sorted(intervals)
    pairs = zip(ordered, ordered[1:])
    return all(a[1] <= b[0] for a, b in pairs)


# FOLLOW-UP 2: Return which room each meeting uses.
# Answer: Keep a heap of (end time, room id): reuse the
# room that frees first, otherwise open the next id, and
# record the id per meeting.
def assign_rooms(intervals):
    # which room does each meeting use?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [None] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
```

```
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# FOLLOW-UP 3: Meetings arrive online.
# Answer: Keep the end-time heap between calls: pop every
# end at or before the new start (those rooms are free),
# then push the new end. The heap size is the number of
# rooms in use right now; the peak is the answer so far.
class RoomTracker:
    """Meetings arrive online (in order of start time)."""

    def __init__(self):
        self.ends, self.peak = [], 0

    def book(self, start, end):
        # rooms whose meeting already ended are free again
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)
        heapq.heappush(self.ends, end)
        # the heap size is the rooms in use right now
        self.peak = max(self.peak, len(self.ends))
        return len(self.ends)


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_meeting_rooms(meetings) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    # touching meetings share a room; empty ones need none
```

```
    touching = [(1, 3), (3, 5), (4, 4)]
    assert min_meeting_rooms(touching) == 1
    assert min_meeting_rooms([]) == 0
    assert not can_attend_all(meetings)
    assert can_attend_all([(7, 10), (2, 4)])
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = RoomTracker()
    in_use = [t.book(s, e) for s, e in meetings]
    assert in_use == [1, 2, 2]
    assert t.peak == 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED · 4 more

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 41 of 167 (Core 90)

Priority score 4.52 | Intervals and scheduling | asked 65 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 3 of 5

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 139/228

ABOUT THIS QUESTION

Topic: Core 90 - Intervals and scheduling

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
# Reverse of LC 986: given two individually sorted,
# internally non-overlapping interval lists, return their
# union. Follow-ups extend to K lists with a heap, inputs
# too large for memory, and lists that may overlap
# internally.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.
#
# BRUTE FORCE (say it first): Concatenate, sort and merge
# Put all intervals in one list, sort by start, and fold
# overlapping ones. Correct and short; the sort ignores
# that both inputs were already sorted. Time O((m + n) log
# (m + n)). Space O(m + n).


import heapq


# BRUTE FORCE (say it first): Concatenate, sort and merge
# | time O((m + n) log (m + n))
def union_sort(a, b):
    out = []
    for s, e in sorted(a + b):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# HELPERS for the base question
# FOLLOW-UP 1: What if the lists are too big for memory?
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))
```

```
# FOLLOW-UP 2: What if one list may overlap itself?
# Answer: The two-pointer merge already works, because it
# folds every incoming interval into the last output
# interval.
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
# unsorted.
# Answer: Normalise each list first (sort and merge it by
# itself, O(n log n)), after which the two-pointer union
# applies. Or sort everything together if memory allows.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
```

```
a = [(1, 3), (6, 8)]
b = [(2, 4), (9, 10)]
assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
# closed intervals that touch merge
assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
# half-open intervals that only touch stay apart
assert union_two([(1, 3)], [(3, 5)], False) == [
    (1, 3), (3, 5)]
# three lists, one of them empty
three = union_k([a, b, []])
assert three == [(1, 4), (6, 8), (9, 10)]
# a list that overlaps itself, and an unsorted one
merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
assert merged == [(1, 12)]
assert union_unsorted([(9, 10), (1, 2)], []) == [
    (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY PATH: question 42 of 167 (Core 90)

Priority score 4.32 | Intervals and scheduling | asked 137 days ago

MEDIUM | PracHub | Technical Screen | 4 of 5

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

POSTED 2026-06-05 | 137 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 40/228**

ABOUT THIS QUESTION

Topic: Core 90 - Intervals and scheduling

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Posted on PracHub: 2026-06-05 (updated 2026-06-14)

Original: https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

APPROACH

Rooms: min-heap of end times. Stream: size-k min-heap, push and pop when over k.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Rooms: is a meeting that starts exactly when another ends a conflict (the scraped solution treats it as no conflict)? Stream: top-K means the K largest values or K most frequent, and what ties do. The scraped solution covers only the rooms part.

BRUTE FORCE TO SAY OUT LOUD

Rooms: assign meetings to rooms and try all orders. Stream: store everything and sort on each query.

OPTIMAL IDEA

Rooms: sort starts and ends separately and sweep with two pointers: if the next start is at least the earliest end, reuse that room, otherwise open a new one; or keep a min-heap of room end times. O(n log n). Stream of the K largest (LeetCode 703): a min-heap of size K, push each value and pop when the size exceeds K, so the root is the K-th largest. O(log K) per value.

DRY-RUN THESE CASES

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

LIKELY FOLLOW-UPS

Return the room assignment, or a sliding window of the last N values for the stream.

PITFALLS

Sorting starts and ends together by start. Returning from the heap before it holds K items.

INTERVIEW-DAY PLAN

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

LIKELY FOLLOW-UPS

Return the room assignment, or a sliding window of the last N values for the stream.

PITFALLS

Sorting starts and ends together by start. Returning from the heap before it holds K items.

HOW TO THINK ABOUT IT

Rooms: the number of rooms needed is the largest number of meetings happening at the same time, so sort the starts and the ends separately and sweep: a start that comes before the earliest end needs a new room. Stream: only the K largest values can matter, so keep exactly those in a min-heap; the root is the K-th largest.

Basis: Two tasks: LeetCode 253 (minimum meeting rooms; back-to-back meetings may share a room) and LeetCode 703 (Kth Largest Element in a Stream).

BRUTE FORCE · Rooms: count the overlap at every time unit

BRUTE FORCE · PSEUDO-CODE

```
def rooms(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE · Stream: sort everything on every query

BRUTE FORCE · PSEUDO-CODE

```
values = []
def add(x):
    values.append(x)
    values.sort(reverse=True)
    return values[K - 1]
```

BRUTE FORCE · Stream: sort everything on every query

TIME	SPACE
O(n log n) per add	O(n)

WHY IT IS SLOW

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION · Rooms: sorted starts and ends with two pointers

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Stream: min-heap of size K

TIME	SPACE
O(log k) per add	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

FOLLOW-UP 2 Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. **Cost:** a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

FOLLOW-UP 3 Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Solve Rooms and Top-K Streams
#
# This question evaluates algorithmic problem-solving
# skills across interval scheduling/resource allocation
# and streaming top-k maintenance, testing competency in
# data structures for ordered queries, tie-breaking logic,
# stateful stream processing, and analysis of time and
# space trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rooms: return which room each meeting uses.
# 2. Stream: K changes over time, or you need the median.
# 3. Rooms: meetings on several floors with capacities.
#
# INTERVIEW NOTES
# Rooms: the number of rooms needed is the largest number
# of meetings happening at the same time, so sort the
# starts and the ends separately and sweep: a start that
# comes before the earliest end needs a new room. Stream:
# only the K largest values can matter, so keep exactly
# those in a min-heap; the root is the K-th largest.
#
# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit
# For each integer time t count the meetings with start <=
# t < end and take the maximum. Correct and the oracle
# (integer times only); it depends on the length of the
# time range, not just n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals),
        )
    )
```

```python
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
```

```
        else:
            rooms += 1
    return rooms


# PART 2: top K values of a stream
class TopKStream:
    """The k largest values seen so far (fixed k)."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def add(self, x):
        # a min-heap of size k: the root is the k-th
        # largest
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)


# VARIANT: median of the stream
```

```
class MedianStream:
    """Follow-up: the median instead of a fixed top-k."""

    def __init__(self):
        # low is a max-heap (negated), high is a min-heap
        self.low, self.high = [], []

    def add(self, x):
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Rooms: return which room each meeting uses.
# Answer: Sort by start and keep a min-heap of (end time,
# room). If the earliest-ending room is free at the next
# start, reuse it, otherwise open a new room. The heap
# size never exceeds the answer.
def assign_rooms(intervals):
    # which room does each meeting use? a heap of (end,
    # room)
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
```

```
    assert min_rooms(meetings) == 2
    # a meeting may start the moment another ends
    assert min_rooms([(1, 3), (3, 5)]) == 1
    assert min_rooms([]) == 0
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        t.add(x)
    assert t.top() == [9, 8, 7]
    m = MedianStream()
    for x in [5, 2, 8, 1]:
        m.add(x)
    assert m.median() == 3.5
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
`([[0, 30], [5, 10], [15, 20]],)`	`2`
`([],)`	`0`

STUDY PATH: question 43 of 167 (Core 90)

Priority score 3.82 | Intervals and scheduling | asked 349 days ago

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 5 of 5

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 41/228

ABOUT THIS QUESTION

Topic: Core 90 - Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2025-11-05 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

QUESTION

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240,
    300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

APPROACH

Max concurrent jobs = servers needed; sweep line over start/end events, or min-heap of end times. Handle intervals wrapping midnight by splitting.

HOW TO THINK ABOUT IT

The number of servers is the largest number of jobs running at the same time. First remove the twist: overlapping intervals of the same job are really one longer interval, so merge the intervals of each job_id. Then it is the plain maximum overlap: sort start and end events and keep a running count.

Basis: LeetCode 253 (Meeting Rooms II) with a twist from the statement: intervals of the same job_id may overlap on one server and count once. Intervals are half-open [start, end) within one day.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def servers_needed(intervals):
    best = 0
    for t in range(DAY):
        jobs = set()
        for job_id, s, e in intervals:
            if s <= t < e:
                jobs.add(job_id)
        best = max(best, len(jobs))
    return best
```

TIME	SPACE
O(T * m)	O(m)

WHY IT IS SLOW

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION · Merge each job's intervals, then sweep start and end events

TIME	SPACE
O(m log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

FOLLOW-UP 2 Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

FOLLOW-UP 3 Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute servers needed for daily
# recurring jobs
#
# You operate a cluster of identical servers that run
# recurring **daily** jobs. For a single day, you are
# given a list of job execution intervals. Each interval
# has: - job_id (string or integer), - start_time
# (timestamp within that day), - end_time (timestamp
# within that day), with start_time < end_time. Interpret
# each interval as a half-open time range [start_time,
# end_time) during which that job must be running on some
# server **on that day**. Rules and constraints: - Each
# physical server can execute at most **one job at a
# time**. - However, if multiple intervals in the input
# correspond to the **same job_id**, they may overlap in
# time **on the same server** (they are considered parts
# or retries of the same job and do not consume extra
# capacity).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Some rare jobs run for several days without restart
# and get a dedicated server.
# 2. Jobs that cross midnight.
# 3. Return which server each job runs on.
#
# INTERVIEW NOTES
# The number of servers is the largest number of jobs
# running at the same time. First remove the twist:
# overlapping intervals of the same job are really one
# longer interval, so merge the intervals of each job_id.
# Then it is the plain maximum overlap: sort start and end
# events and keep a running count.
#
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running
# For each integer time t collect the set of job ids with
# an interval covering t and take the largest set size.
# Correct and the oracle; it depends on the length of the
# day, not only on m. Time O(T * m). Space O(m).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running | time O(T * m)
def servers_by_time(intervals):
```

```
    if not intervals:
        return 0
    lo, hi = min(s for _, s, _ in intervals), max(
        e for _, _, e in intervals
    )
    return max(
        len({j for j, s, e in intervals if s <= t < e})
        for t in range(lo, hi)
    )


# HELPERS for the base question
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks


# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
# FOLLOW-UP 1: Some rare jobs run for several days without
# restart and get a dedicated server.
# Answer: Remove those jobs from the daily intervals and
```

```
# add one server per such job: servers = (number of
# dedicated jobs) + (maximum overlap of the remaining
# jobs).
def servers_needed(intervals, dedicated=()):
    # FOLLOW-UP 1: rare multi-day jobs get their own
    # server
    # dedicated jobs (they span several days) get their
    # own
    # server; the rest share servers: the answer is the
    # largest number of blocks running at the same moment
    dedicated = set(dedicated)
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)


# FOLLOW-UP 2: Jobs that cross midnight.
# Answer: Truncate at the end of the day for each day's
# schedule (as the statement says).
def split_over_midnight(start, end, day=24 * 60):
```

```
    # a daily job that wraps past midnight: two pieces
    if start < end:
        return [(start, end)]
    return [(start, day), (0, end)]


# FOLLOW-UP 3: Return which server each job runs on.
# Answer: Sort merged intervals by start and keep a min-
# heap of (end time, server id): reuse the server that
# frees up first if it is free, otherwise open a new one.
# The number of servers opened equals the maximum overlap.
def assign_servers(intervals):
    # which server runs each block of each job?
    heap, plan, opened = [], [], 0
    blocks = sorted(merge_by_job(intervals))
```

```
    for start, end, job in blocks:
        if heap and heap[0][0] <= start:
            _, server = heapq.heappop(heap)
        else:
            server, opened = opened, opened + 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened


if __name__ == "__main__":
    jobs = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8),
            ("c", 10, 15)]
    # job a is one block [0, 12); b overlaps it; c starts
    # at 10
    assert servers_needed(jobs) == 2
    # touching half-open intervals share a server
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1
    assert servers_needed([]) == 0
    # job b runs for days: it gets a dedicated server
    assert servers_needed(jobs, dedicated=["b"]) == 3
    assert servers_needed(jobs, dedicated=["a", "b"]) == 3
    plan, count = assign_servers(jobs)
    assert count == 2 and len(plan) == 3
    assert split_over_midnight(1380, 60) == [(1380, 1440),
                                             (0, 60)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED · 1 more

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks